```
    const llm = new FakeLLM({
      response: 'The answer based on the provided context.',
    });

    ragSystem.retriever = retriever;
    ragSystem.llm = llm;
  });

  it('should throw an error if retriever is not
initialized', async () => {
    ragSystem.retriever = null;

    await expect(ragSystem.performRAG('What is AI?')).rejects.
toThrow('Retriever not initialized.
Run indexDocuments first.');
  });

  it('should return an answer and source documents',
async () => {
    const query = 'What is AI?';

    const result = await ragSystem.performRAG(query);

    expect(result.answer).toEqual('The answer based
on the provided context.');
    expect(result.sourceDocuments).toEqual([
      { pageContent: 'This is some context about AI.' },
      { pageContent: 'This is another context about machine learning.' },
    ]);
  });
});
```

Sets up a FakeLLM with a fixed response to simulate AI-generated answers

Tests the error handling when the retriever is not initialized, ensuring the system behaves correctly

Tests the main functionality of the RAG system, verifying that it returns the expected answer and source documents

Here, the code creates mock instances of `FakeRetriever` and `FakeLLM`, which simulate document retrieval and AI responses, respectively. The first test case verifies that an error is thrown when the retriever is not initialized, ensuring proper error handling. The second test case checks that the `performRAG` method returns the expected answer and source documents when given a valid query.

Given that the class `RAGSystem` that we created contains instances of a retriever and LLM as properties, can we refactor our code so it's easier to test? The answer is, of course, yes—if it makes sense to make your code more testable and composable in nature and it's a valuable testing strategy. One way to refactor this is to use dependency injection. By injecting dependencies such as the retriever and LLM into the class rather than hardcoding them as properties, we can create a more flexible and modular design. This approach allows for easier substitution of real implementations with mock objects during testing.

The following is a short code snippet demonstrating how to refactor the `RAGSystem` class to use dependency injection:

```
class RAGSystem {
  constructor(apiKey, retriever, llm) {
```